

Technical Documentation

Optimized Container Orchestration for Web Applications

2º DAW – Despliegue de Aplicaciones Web | RA3

Date: May 2026
Student: María Abascal Jiménez
Repository: <https://github.com/mabajim065/docker-web-app-ra3>

Architecture Overview

This project implements a three-tier architecture using Docker and Docker Compose, with four independent services:

Service	Technology	Role	Network
nginx	Nginx Alpine	Reverse proxy – single entry point	frontend-network
frontend	React + Nginx Alpine	User interface layer	frontend-network
backend	Spring Boot + JRE Alpine	Business logic and API	frontend-network + backend-network
db	MySQL 8.0	Persistent data storage	backend-network

Network Isolation:

Two isolated Docker bridge networks are used. The **backend-network** connects only the backend and the database, so the database is never directly accessible from nginx or the frontend. The **frontend-network** connects nginx, frontend, and backend.

Deployment Manual

Prerequisites

- Docker Desktop installed and running
- Git installed

Steps to Deploy

1. **Clone the repository**
git clone https://github.com/mabajim065/docker-web-app-ra3.git
cd docker-web-app-ra3
2. **Create the .env file**
cp .env.example .env
3. **Start all services**
docker compose up -d --build
4. **Access the application**
Open browser at: <http://localhost>
5. **Stop all services**
docker compose down
6. **Update a service**
docker compose build backend
docker compose up -d backend

Configuration Files

File	Description
docker-compose.yml	Orchestrates all 4 services, networks, volumes, healthchecks and resource limits.
.env	Environment variables with sensitive data (not committed to Git)
nginx/nginx.conf	Reverse proxy routing: / → frontend, /api → backend
frontend/Dockerfile	Multi-stage build: Node.js builder → Nginx Alpine production
backend/Dockerfile	Multi-stage build: Maven builder → JRE Alpine production
frontend/nginx.conf	Internal Nginx config for React Router
db_backup.sh	Automated backup script using docker exec + mysqldump

Multi-Stage Builds

Both Dockerfiles use multi-stage builds to produce optimized, secure production images.

Frontend (React)

- **Stage 1 (builder):** Uses node:18-alpine to install npm dependencies and run the React build.
- **Stage 2 (production):** Uses nginx:alpine and copies only the compiled static files from the builder.

The final image does NOT contain Node.js, npm, or any source code.

Backend (Spring Boot)

- **Stage 1 (builder):** Uses maven:3.9-eclipse-temurin-17-alpine to compile the Java project and package it as a .jar.
- **Stage 2 (production):** Uses eclipse-temurin:17-jre-alpine and copies only the compiled .jar.

Maven, source code, and build tools are completely absent from the final image.

Security benefit: smaller images have fewer packages, fewer vulnerabilities, and a smaller attack surface. Build secrets and intermediate files never reach production.

Healthchecks and Startup Order

MySQL takes several seconds to fully initialize after the container starts. Without a healthcheck, the backend would try to connect before the database is ready, causing connection errors and crashes.

The solution implemented:

- The db service has a healthcheck that runs mysqladmin ping every 10 seconds with a 30-second start period.
- The backend service uses depends_on with condition: service_healthy, so Docker strictly waits for the database healthcheck to pass before starting Spring Boot.

This completely prevents race conditions during initial deployment.

Administration Manual

Managing Services

```
docker compose ps          # View running containers
docker compose logs backend # View backend logs
docker compose logs nginx   # View nginx logs
docker compose restart backend # Restart a service
docker compose down         # Stop all services
docker stats                # Monitor resource usage
```

Security Measures Applied

Measure	Implementation
Minimal exposed ports	Only port 80 (nginx) exposed to host
Environment variables	All credentials stored in .env, excluded from Git

Measure	Implementation
Non-root users	Containers run as appuser (non-root)
Isolated networks	DB only reachable from backend via backend-network
Pinned image versions	mysql:8.0, node:18-alpine, nginx:alpine, etc.
Resource limits	512MB RAM and 0.5 CPU for backend and db

Database Backup

Run the backup script from the project root:

```
bash db_backup.sh
```

The script connects to the db container using docker exec, runs mysqldump inside the container, and saves a timestamped .sql file in ./backups/ on the host.

Maintenance and Scalability Recommendations

- Run db_backup.sh regularly and store backups safely
- Monitor with: docker stats
- Update base images periodically for security patches
- Use strong passwords in production
- For higher availability, consider Docker Swarm or Kubernetes
- Add SSL/TLS with HTTPS in production environments

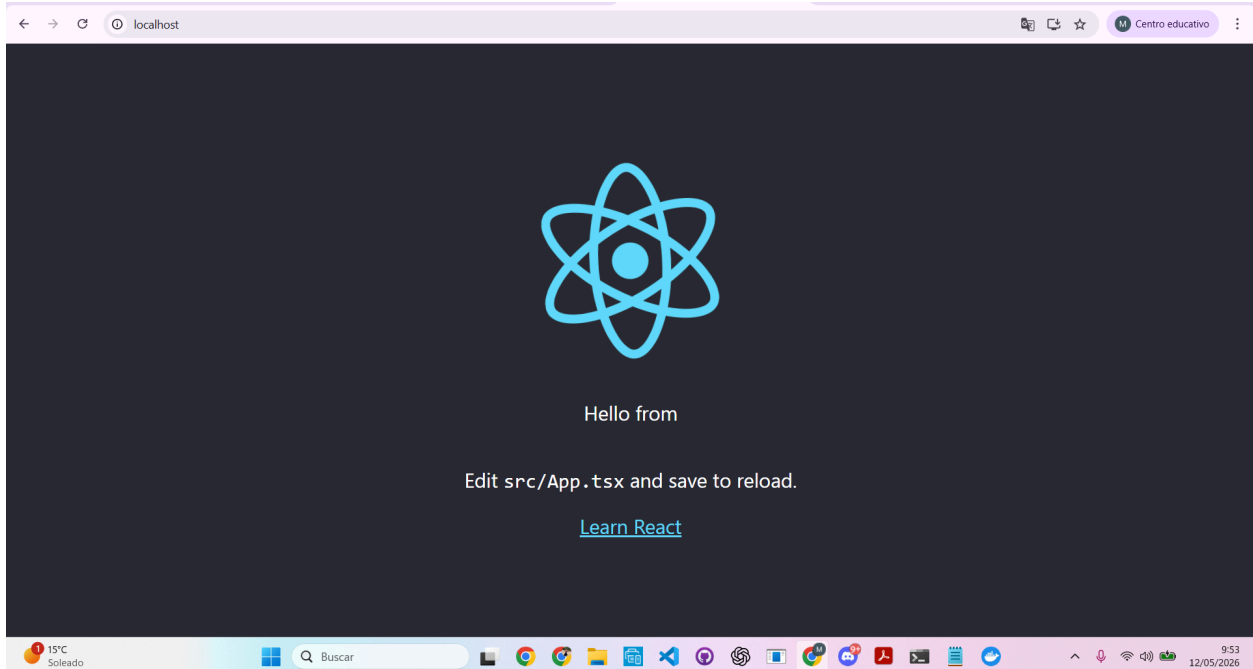
Testing and Validation

Test 1 – All containers running

```
C:\Users\juanc\docker-web-app-ra3>docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS                PORTS
backend             docker-web-app-ra3-backend  "java -jar app.jar"    backend    4 minutes ago  Up 4 minutes         8080/tcp
db                  mysql:8.0            "docker-entrypoint.s..." db         4 minutes ago  Up 4 minutes (healthy)  3306/tcp, 33060/tcp
frontend            docker-web-app-ra3-frontend  "/docker-entrypoint...." frontend   4 minutes ago  Up 4 minutes         80/tcp
nginx               docker-web-app-ra3-nginx    "/docker-entrypoint...." nginx      4 minutes ago  Up 4 minutes         0.0.0.0:80->80/tcp, [::]:80->80/tcp
```

Caption: All 4 containers running. DB status shows (healthy). Only nginx exposes port 80 to the host.

Test 2 – Frontend accessible via Nginx



Caption: React frontend loading correctly through Nginx at http://localhost

Test 3 – Backend logs (Spring Boot startup)

```
C:\Users\juanc\docker-web-app-ra3>docker compose logs backend
backend
backend
backend
backend
backend
backend
:: Spring Boot :: (v2.2.5.RELEASE)
2026-05-12 07:50:23.441 INFO 1 --- [main] com.company.project.Application : Starting Application v0.0.1-SNAPSHOT on 21219dda28f3 with PID 1 (/app/app.jar st
arted by appuser in /app)
2026-05-12 07:50:23.448 INFO 1 --- [main] com.company.project.Application : No active profile set, falling back to default profiles: default
2026-05-12 07:50:30.852 INFO 1 --- [main] .s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA repositories in DEFAULT mode.
2026-05-12 07:50:31.454 INFO 1 --- [main] .s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository scanning in 495ms. Found 1 JPA repository interf
aces.
2026-05-12 07:50:37.941 INFO 1 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port(s): 8080 (http)
2026-05-12 07:50:37.962 INFO 1 --- [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-05-12 07:50:37.962 INFO 1 --- [main] org.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/9.0.31]
2026-05-12 07:50:38.542 INFO 1 --- [main] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2026-05-12 07:50:38.542 INFO 1 --- [main] o.s.web.context.ContextLoader : Root WebApplicationContext: initialization completed in 14685 ms
2026-05-12 07:50:41.144 INFO 1 --- [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting...
2026-05-12 07:50:44.353 INFO 1 --- [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed.
2026-05-12 07:50:46.750 INFO 1 --- [main] org.hibernate.jpa.internal.util.LogHelper : HHH000204: Processing PersistenceUnitInfo [name: default]
2026-05-12 07:50:47.551 INFO 1 --- [main] org.hibernate.Version : HHH000412: Hibernate ORM core version 5.4.12.Final
2026-05-12 07:50:49.341 INFO 1 --- [main] o.hibernate.annotations.common.Version : HCANN000001: Hibernate Commons Annotations {5.1.0.Final}
2026-05-12 07:50:50.404 INFO 1 --- [main] org.hibernate.dialect.Dialect : HHH000408: Using dialect: org.hibernate.dialect.MySQL8Dialect
2026-05-12 07:50:57.035 INFO 1 --- [main] o.h.e.t.j.p.i.JtaPlatformInitiator : HHH000490: Using JtaPlatform implementation: [org.hibernate.engine.transaction.j
ta.platform.internal.NoJtaPlatform]
2026-05-12 07:50:57.056 INFO 1 --- [main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
2026-05-12 07:50:59.837 WARN 1 --- [main] JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, database queries may b
e performed during view rendering. Explicitly configure spring.jpa.open-in-view to disable this warning
2026-05-12 07:51:00.548 INFO 1 --- [main] o.s.s.concurrent.ThreadPoolTaskExecutor : Initializing ExecutorService 'applicationTaskExecutor'
2026-05-12 07:51:02.552 INFO 1 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port(s): 8080 (http) with context path ''
2026-05-12 07:51:02.636 INFO 1 --- [main] com.company.project.Application : Started Application in 43.379 seconds (JVM running for 48.453)
2026-05-12 07:52:08.029 INFO 1 --- [nio-8080-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
2026-05-12 07:52:08.030 INFO 1 --- [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2026-05-12 07:52:08.131 INFO 1 --- [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 101 ms
C:\Users\juanc\docker-web-app-ra3>
```

Caption: Spring Boot started on port 8080, connected to MySQL. Running as appuser.

Test 4 – Nginx logs (request routing)

```

C:\Users\juanc\docker-web-app-ra3>docker compose logs nginx
nginx | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
nginx | /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
nginx | /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
nginx | 10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
nginx | 10-listen-on-ipv6-by-default.sh: info: /etc/nginx/conf.d/default.conf differs from the packaged version
nginx | /docker-entrypoint.sh: Saucing /docker-entrypoint.d/15-local-resolvers.envsh
nginx | /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
nginx | /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
nginx | /docker-entrypoint.sh: Configuration complete; ready for start up
nginx | 2026/05/12 07:50:14 [notice] 1#1: using the "epoll" event method
nginx | 2026/05/12 07:50:14 [notice] 1#1: nginx/1.29.5
nginx | 2026/05/12 07:50:14 [notice] 1#1: built by gcc 15.2.0 (Alpine 15.2.0)
nginx | 2026/05/12 07:50:14 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
nginx | 2026/05/12 07:50:14 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
nginx | 2026/05/12 07:50:14 [notice] 1#1: start worker processes
nginx | 2026/05/12 07:50:14 [notice] 1#1: start worker process 30
nginx | 2026/05/12 07:50:14 [notice] 1#1: start worker process 31
nginx | 2026/05/12 07:50:14 [notice] 1#1: start worker process 32
nginx | 2026/05/12 07:50:14 [notice] 1#1: start worker process 33
nginx | 2026/05/12 07:50:14 [notice] 1#1: start worker process 34
nginx | 2026/05/12 07:50:14 [notice] 1#1: start worker process 35
nginx | 2026/05/12 07:50:14 [notice] 1#1: start worker process 36
nginx | 2026/05/12 07:50:14 [notice] 1#1: start worker process 37
nginx | 172.23.0.1 -- [12/May/2026:07:52:07 +0000] "GET / HTTP/1.1" 200 644 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36" "-"
nginx | 172.23.0.1 -- [12/May/2026:07:52:07 +0000] "GET /static/css/main.073c9b0a.css HTTP/1.1" 200 1044 "http://localhost/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36" "-"
nginx | 172.23.0.1 -- [12/May/2026:07:52:07 +0000] "GET /manifest.json HTTP/1.1" 200 517 "http://localhost/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36" "-"
nginx | 172.23.0.1 -- [12/May/2026:07:52:07 +0000] "GET /static/js/main.9beff93e.js HTTP/1.1" 200 133229 "http://localhost/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36" "-"
nginx | 172.23.0.1 -- [12/May/2026:07:52:07 +0000] "GET /static/media/logo.2d27ead7c10f774c5306bcf3c257a728.svg HTTP/1.1" 200 2678 "http://localhost/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36" "-"
nginx | 172.23.0.1 -- [12/May/2026:07:52:07 +0000] "GET /favicon.ico HTTP/1.1" 200 3150 "http://localhost/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36" "-"
nginx | 172.23.0.1 -- [12/May/2026:07:52:07 +0000] "GET /logo192.png HTTP/1.1" 200 5347 "http://localhost/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36" "-"
nginx | 172.23.0.1 -- [12/May/2026:07:52:09 +0000] "GET /api HTTP/1.1" 404 135 "http://localhost/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36" "-"
C:\Users\juanc\docker-web-app-ra3>

```

Caption: Nginx processing requests. All frontend static files return 200 OK. /api routed to backend.

Test 5 – Reverse proxy routing to backend

Whitelabel Error Page

This application has no explicit mapping for /error, so you are seeing this as a fallback.

Tue May 12 08:05:50 GMT 2026

There was an unexpected error (type=Not Found, status=404).

No message available

Caption: Spring Boot Whitelabel Error confirms Nginx correctly routed /api to the backend. The 404 is expected — the reference app has no root /api endpoint, but the proxy routing works correctly.

Test 6 – Automated database backup

```
MINGW64:/c/Users/juanc/docker-web-app-ra3
juanc@LAPTOP-OVILC63U MINGW64 ~
$ cd /c/Users/juanc/docker-web-app-ra3
juanc@LAPTOP-OVILC63U MINGW64 ~/docker-web-app-ra3 (master)
$ bash db_backup.sh
Starting backup of database 'appdb'...
mysqldump: [Warning] Using a password on the command line interface can be insecure.
Backup completed successfully: ./backups/backup_20260512_102425.sql
juanc@LAPTOP-OVILC63U MINGW64 ~/docker-web-app-ra3 (master)
$
```

Caption: db_backup.sh executed successfully. backup_20260512_102425.sql saved in ./backups/

Test Summary

Test	Method	Result
Frontend accessible via Nginx	Browser → http://localhost	PASS
Backend reachable via /api proxy	Browser → http://localhost/api	PASS
Backend connects to database	docker compose logs backend	PASS
Database healthcheck	docker compose ps	PASS – healthy
Only nginx exposes ports	docker compose ps	PASS – only 0.0.0.0:80
Automated backup script	bash db_backup.sh	PASS
Multi-stage build frontend	docker images	PASS – nginx:alpine base
Multi-stage build backend	docker images	PASS – jre-alpine base